Sam Freund

Lab 11

CSC 2611

Part 1

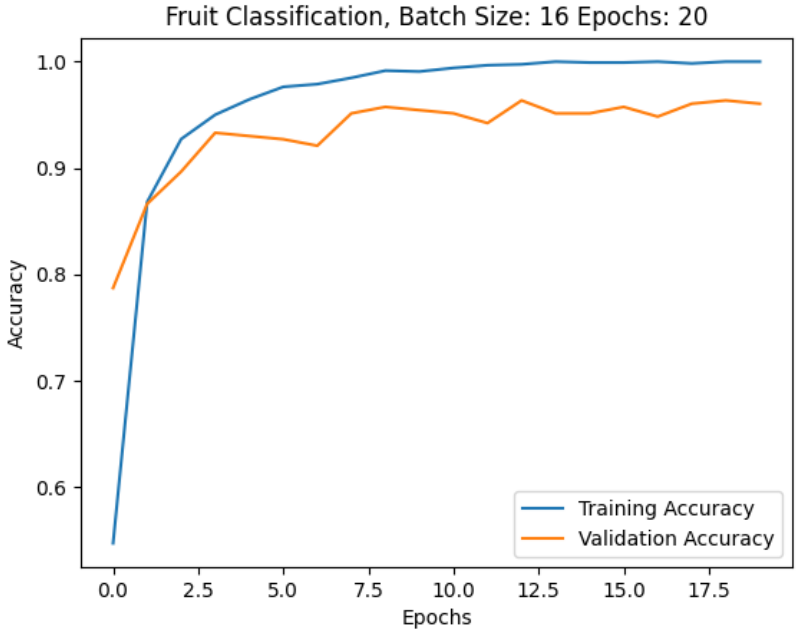
VGG16 Overview

VGG16 is a deep convolutional neural network developed by the Visual Geometry Group at Oxford, known for its simple, uniform architecture composed of stacked 3×3 convolutional layers and 2×2 max-pooling layers. It has 16 weight layers and was designed to test how increasing network depth affects performance. VGG16 was trained on the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) dataset, which contains over 1.2 million labeled images across 1,000 object categories. As a result, VGG16 learns rich, hierarchical visual features—edges, textures, shapes, and object parts—making it effective for image classification and useful as a feature extractor for transfer learning.

srun

```
Namespace(augment_data='true', batch_size='20', data='/data/cs2300/L9/fruits', epochs='16',
Found 1182 images belonging to 6 classes.
Found 329 images belonging to 6 classes.
Epoch 1/16
60/59 - 31s - loss: 1.9712 - accuracy: 0.5288 - val_loss: 0.6461 - val_accuracy: 0.7933
Epoch 2/16
60/59 - 28s - loss: 0.4806 - accuracy: 0.8342 - val_loss: 0.4018 - val_accuracy: 0.8693
Epoch 3/16
60/59 - 25s - loss: 0.2612 - accuracy: 0.9120 - val_loss: 0.2224 - val_accuracy: 0.9331
Epoch 4/16
60/59 - 26s - loss: 0.1744 - accuracy: 0.9391 - val_loss: 0.2383 - val_accuracy: 0.9149
Epoch 5/16
60/59 - 27s - loss: 0.1319 - accuracy: 0.9569 - val_loss: 0.1940 - val_accuracy: 0.9422
Epoch 6/16
60/59 - 26s - loss: 0.0940 - accuracy: 0.9738 - val_loss: 0.1813 - val_accuracy: 0.9483
Epoch 7/16
60/59 - 27s - loss: 0.0771 - accuracy: 0.9763 - val_loss: 0.1674 - val_accuracy: 0.9605
Epoch 8/16
60/59 - 26s - loss: 0.0576 - accuracy: 0.9882 - val_loss: 0.1314 - val_accuracy: 0.9635
Epoch 9/16
60/59 - 27s - loss: 0.0453 - accuracy: 0.9915 - val_loss: 0.1925 - val_accuracy: 0.9544
Epoch 10/16
```

```
60/59 - 26s - loss: 0.0389 - accuracy: 0.9907 - val_loss: 0.1325 - val_accuracy: 0.9696
Epoch 11/16
60/59 - 26s - loss: 0.0313 - accuracy: 0.9966 - val_loss: 0.1377 - val_accuracy: 0.9605
Epoch 12/16
60/59 - 26s - loss: 0.0283 - accuracy: 0.9958 - val_loss: 0.1496 - val_accuracy: 0.9605
Epoch 13/16
60/59 - 26s - loss: 0.0242 - accuracy: 0.9966 - val_loss: 0.1262 - val_accuracy: 0.9635
Epoch 14/16
60/59 - 27s - loss: 0.0226 - accuracy: 0.9983 - val_loss: 0.1410 - val_accuracy: 0.9696
Epoch 15/16
60/59 - 26s - loss: 0.0183 - accuracy: 1.0000 - val_loss: 0.1535 - val_accuracy: 0.9574
Epoch 16/16
60/59 - 25s - loss: 0.0175 - accuracy: 0.9992 - val_loss: 0.1301 - val_accuracy: 0.9696
```



Part 2

42 accurate predictions out of 50 images for freshapples. Accuracy: 0.84

48 accurate predictions out of 49 images for freshbanana. Accuracy: 0.979

42 accurate predictions out of 43 images for freshoranges. Accuracy: 0.976

68 accurate predictions out of 74 images for rottenapples. Accuracy: 0.918

67 accurate predictions out of 68 images for rottenbanana. Accuracy: 0.985

17 accurate predictions out of 45 images for rottenoranges. Accuracy: 0.377

Part 3

I find this to be much simpler than typing out the entire srun command. However, this is (in my opinion) a bit less persistent as the history of the `.sh` file isn't tracked, while the previous srun command can be found using `history`.

Running `lab11.sh`, we use hyperparameters batch_size=32 and epochs=5. Results of the final epoch can be found below.
`37/36 - 25s - loss: 0.1809 - accuracy: 0.9349 - val_loss: 0.2246 - val_accuracy: 0.9179`

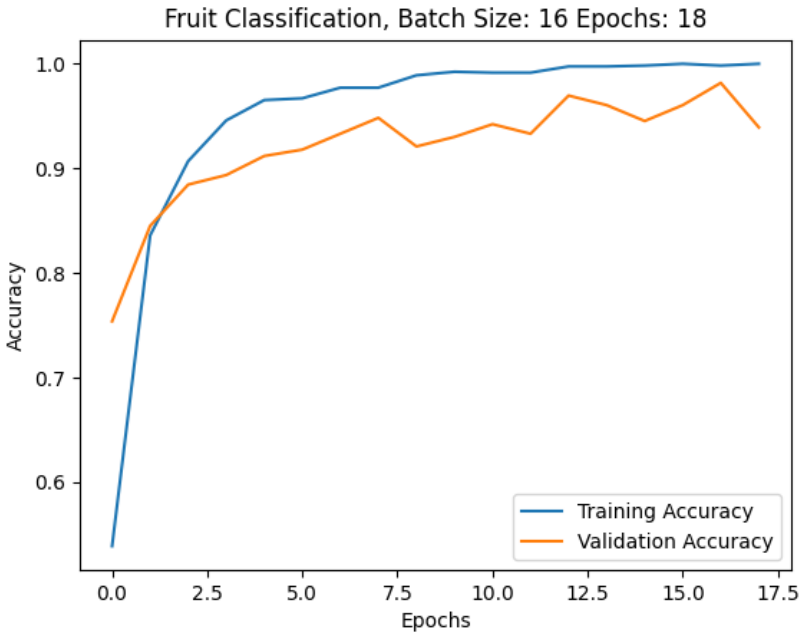
Part 4

For testing different parameters, I used a grid search running on a slurm array, implemented with the below script.

```
#!/bin/bash
#SBATCH --partition=teaching
#SBATCH --nodes=1
#SBATCH --gpus=1
#SBATCH --cpus-per-gpu=20
#SBATCH --error='sbatcherrorfile_%A_%a.out'
#SBATCH --output='sbatchoutputfile_%A_%a.out'
#SBATCH --time=0-2:0
# Array job setup - adjust based on grid search size
#SBATCH --array=0-11

####
# Grid search parameters
####
# Define batch sizes to search over
batch_sizes=(16 32 64 128 256)
# Define epochs to search over
epochs_values=(5 10 15 20)

# Calculate total combinations
num_batch_sizes=${#batch_sizes[@]}
num_epochs=${#epochs_values[@]}

# Map array task ID to parameter combination
batch_idx=$((SLURM_ARRAY_TASK_ID / num_epochs))
epoch_idx=$((SLURM_ARRAY_TASK_ID % num_epochs))

# Get current parameters
current_batch_size=${batch_sizes[$batch_idx]}
```

```
current_epochs=${epochs_values[$epoch_idx]}

echo "Running experiment with batch_size=$current_batch_size and epochs=$current_epochs"
echo "Job ID: $SLURM_ARRAY_JOB_ID, Task ID: $SLURM_ARRAY_TASK_ID"

####
# Job execution
####
# Path to container
container="/data/containers/msoe-tensorflow-20.07-tf2-py3.sif"

# Command to run inside container
command="python Lab11.py --data /data/cs2300/L9/fruits --batch_size $current_batch_size --ep

# Execute singularity container on node
singularity exec --nv -B /data:/data ${container} /usr/local/bin/nvidia_entrypoint.sh ${comm
```

The first set of runs I tested batch sizes up to 128 and epochs up to 15. I found that this did not result in high enough accuracy, so I added another value to each list and reran. I found that increasing epochs helped me get closer, but increasing batch size was not helpful. The best value from that run was 0.972, using a batch size of 16 and running 20 epochs. The third run used batch sizes of 12, 16, 20, and 24; with epochs 14, 16, 18, 20. With this batch, I found that using a batch size of 16 and 18 epochs I was able to achieve an accuracy of 0.981 on the second to last epoch.


Fruit Classification, Batch Size: 16 Epochs: 18
Accuracy
Epochs
Training Accuracy
Validation Accuracy
1.0
0.9
0.8
0.7
0.6
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5